

Université Clermont Auvergne

Clermont Auvergne INP – ISIMA

Institut Supérieur d'Informatique,
de Modélisation et de leurs Applications

Campus Universitaire des Cézeaux
1 Rue de la Chebarde – TSA 60026
63178 Aubière cedex

Rapport de TER

Première année de Master Intelligence Artificielle

Année universitaire 2025–2026

Développement d'un prototype RAG local pour l'analyse de documents techniques

Projet EVOLUTYS

Présenté par :

Khemira Seif Eddine

Tuteurs académique

Engelbert Mephu Nguifo

engelbert.mephu_nguifo@uca.fr

Boris Nectoux

boris.nectoux@uca.fr

Nourddine Azzaoui

nourddine.azzaoui@uca.fr

Table des matières

Contexte et portée	2
1 Résolution des conflits d’environnement	3
2 Pipeline initial d’ingestion et de traitement	3
2.1 Ingestion et découpage	3
2.2 Formatage des extraits	3
2.3 Instructions de réponse	3
2.4 Représentation vectorielle et rôle de Chroma	4
3 Comparaison des recherches sémantique et lexicale : le test VULCASTEEL	4
4 Premiers tests et limites de la recherche	4
5 Passage du terminal à Streamlit	5
6 Implémentation d’un moteur de recherche hybride personnalisé	5
7 Introduction de la recherche parent/enfant	6
8 Contrôle de la quantité de contexte	6
9 Ajout de la fusion de rangs réciproques	7
10 Ajout des noms de sources et de la pagination	7
11 Migration de Mistral vers Qwen3	8
11.1 Construction du prompt technique	8
12 Ajout d’un reranker neuronal	9
12.1 Sélection des parents et limitation de la redondance	9
12.2 Gestion des ressources matérielles	10
13 Extension du prototype à plusieurs PDF	10
13.1 Gestion des changements du corpus	11
13.2 Architecture de traitement d’une question	11
14 Évaluation automatique avec RAGAS	12
14.1 Résultats	13
14.2 Analyse des réponses individuelles	14
14.3 Limites du juge local	14
15 Améliorations finales de l’interface	15
15.1 Conservation des réponses pendant la session	15
15.2 Limites techniques et améliorations envisagées	15
16 Conclusion et perspectives	16

Contexte et portée

Ce projet porte sur le développement d'une application locale de génération augmentée par la recherche (*Retrieval-Augmented Generation*, RAG) pour l'analyse de documents techniques, notamment les CCTP relatifs aux activités industrielles et aux ICPE (Installations Classées pour la Protection de l'Environnement).

Il répond aux besoins d'un bureau d'études en ingénierie en traitant les documents et en exécutant les modèles de langage localement sur un ordinateur Apple Silicon. Le téléchargement initial des modèles et l'installation des dépendances nécessitent un accès à des dépôts externes.

Ce rapport décrit les étapes successives du développement, depuis un prototype mono-document jusqu'à une application multi-documents comprenant deux configurations RAG. Les outils, paramètres et limites décrits dans les premières sections correspondent aux versions utilisées à ces étapes.

Objectif et périmètre du prototype

Le besoin initial d'EVOLUTYS couvrait plusieurs usages de l'intelligence artificielle : analyse de cahiers des charges, extraction d'informations techniques et génération assistée de documents. Dans le cadre de ce TER, le travail a été concentré sur une première fonctionnalité : interroger des documents PDF en langage naturel et fournir une réponse accompagnée de passages sources.

Le prototype devait permettre de charger plusieurs documents, de retrouver des informations techniques précises et de rendre les réponses vérifiables. Le traitement local constituait une contrainte importante pour limiter l'exposition des documents à des services externes.

La génération de mémoires techniques complets et l'intégration à un système d'information géographique ne font pas partie des fonctionnalités réalisées. Le projet constitue une première étape de faisabilité, centrée sur la recherche documentaire et l'évaluation des réponses.

La question directrice du travail peut être formulée ainsi : *dans quelle mesure une architecture RAG locale peut-elle retrouver des informations techniques dans plusieurs CCTP et produire des réponses vérifiables, avec un temps de traitement compatible avec une utilisation interactive ?*

Différence entre indexation et entraînement

Aucun entraînement supplémentaire des modèles n'a été réalisé sur les CCTP. Les documents sont indexés, puis des passages sont fournis au modèle au moment de chaque question.

L'ajout d'un PDF modifie le corpus interrogeable, mais pas les paramètres appris du modèle génératif. L'indexation documentaire doit donc être distinguée d'un ajustement du modèle par entraînement, souvent appelé *fine-tuning*.

L'ajout de nouveaux PDF permet d'examiner le comportement du système sur un corpus plus varié. Il ne constitue pas, à lui seul, un test de surapprentissage. Une évaluation plus solide nécessiterait notamment des questions et des documents qui n'ont pas servi à ajuster les choix du prototype.

1 Résolution des conflits d'environnement

La première étape du développement a consisté à isoler l'environnement Python du projet afin d'éviter les conflits entre les dépendances.

Les commandes d'installation et d'exécution ciblent spécifiquement l'environnement virtuel du projet :

```
./venv/bin/pip install <paquet>  
./venv/bin/python <script.py>
```

Ces chemins sont relatifs au répertoire du projet et garantissent l'utilisation de l'interpréteur Python et des dépendances ciblés.

La chaîne de traitement a été assemblée à l'aide de **LCEL (LangChain Expression Language)** pour relier le prompt, le modèle de langage et l'analyseur de sortie. Cela a réduit la dépendance aux composants de plus haut niveau dont la disponibilité variait selon les versions de la bibliothèque.

2 Pipeline initial d'ingestion et de traitement

Le premier prototype traitait un seul PDF. Les annotations du script expliquaient le rôle de chaque étape.

2.1 Ingestion et découpage

Avec `PDFPlumberLoader()`, le PDF sélectionné était chargé sous forme d'objets document contenant le texte extrait et des métadonnées.

Le découpage était réalisé avec `RecursiveCharacterTextSplitter()`. Il permettait de diviser le texte en passages de taille limitée avant la recherche. Cette opération est distincte du formatage des passages récupérés.

2.2 Formatage des extraits

Dans la première version, `format_docs()` concaténait les extraits en un texte lisible, avec des sauts de ligne explicites.

Les versions ultérieures ont intégré cette étape sous la forme de `formater_contexte()`, qui inclut les noms des documents et les références de pages.

2.3 Instructions de réponse

Le prompt système indiquait au modèle de langage de répondre uniquement à partir du contexte fourni et de reconnaître lorsque l'information n'était pas disponible.

Cela limitait la possibilité de réponses non étayées, sans garantir leur exactitude factuelle. Les tests ultérieurs ont révélé la nécessité d'améliorer la récupération des passages et de procéder à une évaluation explicite.

2.4 Représentation vectorielle et rôle de Chroma

La recherche sémantique repose sur une représentation numérique des passages, appelée embedding. Le modèle d’embeddings transforme le texte en vecteurs utilisés pour rechercher des passages proches de la question dans cet espace de représentation.

Dans l’application, Chroma assure le stockage et la recherche de ces représentations. Le modèle d’embeddings, la base vectorielle et le modèle génératif ont des rôles différents.

TABLE 1 – Rôle des composants de la recherche sémantique.

Composant	Rôle
Modèle d’embeddings	Transformer les textes en représentations numériques.
Chroma	Stocker les vecteurs et rechercher des passages candidats.
Métadonnées	Conserver le document, la page et le lien enfant–parent.
Modèle génératif	Rédiger une réponse à partir des passages sélectionnés.

Dans la version parent/enfant développée ensuite, seuls les fragments enfants sont indexés pour la recherche. Leurs métadonnées permettent de retrouver les parents correspondants, conservés séparément, afin de transmettre un contexte plus large au modèle génératif.

Le code actuel ne configure pas de répertoire de persistance pour Chroma. La réutilisation des ressources repose sur le cache de l’application ; elle ne doit pas être confondue avec une sauvegarde durable de l’index permettant sa réutilisation après un redémarrage complet.

3 Comparaison des recherches sémantique et lexicale : le test VULCASTEEL

Un premier test portait sur des spécifications techniques relatives à la marque **VULCASTEEL**.

Recherche sémantique

La recherche utilisant `nomic-embed-text` n’a pas renvoyé les passages nécessaires pour répondre à la question. Cela suggérait que la similarité sémantique seule ne suffisait pas à retrouver cet identifiant technique particulier dans la configuration initiale.

Recherche lexicale

BM25 a retrouvé le nom de la marque et les classifications techniques associées, notamment **A2-s1-d0**, **EI120** et **Broof t3**.

Cette observation a motivé l’introduction d’une recherche hybride afin de combiner la recherche sémantique et la correspondance lexicale.

Elle n’établit pas pour autant que les modèles d’embeddings sont généralement incapables de représenter des noms de marques.

4 Premiers tests et limites de la recherche

Deux types de questions ont été utilisés pour tester le système initial.

Test 1 : recherche d'informations ciblée

Le prototype a identifié une contrainte de surface associée au local étuves.

Test 2 : question combinant la gestion des déchets et les EPI

Lorsqu'une question combinait la gestion des déchets et les équipements de protection individuelle, le contexte récupéré n'incluait pas le passage critique sur les EPI mentionnant les casques, les gants et les chaussures de sécurité.

L'hypothèse était que les passages associés à la gestion des déchets dominaient le classement de la recherche. La raison exacte n'a cependant pas été isolée : la formulation de la requête, les limites des fragments, la tokenisation et le nombre de passages sélectionnés auraient pu y contribuer.

Le constat était donc qu'une **question multi-sujets pouvait recevoir un contexte incomplet**.

5 Passage du terminal à Streamlit

Le prototype fonctionnait initialement en ligne de commande. Une interface locale **Streamlit**, enrichie d'un CSS personnalisé, a ensuite été développée pour faciliter la démonstration et l'utilisation.

L'interface a ajouté :

- un champ de saisie de questions et un bouton d'analyse ;
- une zone de réponse lisible ;
- des panneaux déroulants exposant les passages récupérés ;
- des informations sur la configuration actuelle.

La mise en cache via `@st.cache_resource` a été introduite progressivement pour réutiliser les ressources préparées.

Dans la première version, la préparation des documents était mise en cache, tandis que les composants de recherche étaient reconstruits à chaque question. Les versions ultérieures ont mis en cache l'index préparé et les ressources du modèle.

Cette étape a transformé le script expérimental en un prototype interactif adapté à une démonstration en bureau d'études.

6 Implémentation d'un moteur de recherche hybride personnalisé

L'étape suivante a combiné la recherche lexicale BM25 avec la recherche par similarité vectorielle.

Une tentative d'intégration utilisant **EnsembleRetriever** de LangChain a échoué en raison d'un problème de compatibilité dans l'environnement installé. Une fonction Python personnalisée a donc été implémentée pour fusionner les résultats récupérés.

La première implémentation se contentait de concaténer les résultats lexicaux et sémantiques, de supprimer les références parentes en double et de sélectionner un nombre limité de parents.

C’était une solution intermédiaire utile, mais elle présentait une limite : les résultats apparaissant plus tôt dans la liste fusionnée pouvaient être favorisés sans classement conjoint de pertinence.

Cela a motivé l’introduction ultérieure de la **fusion de rangs réciproques (RRF)**.

7 Introduction de la recherche parent/enfant

Une stratégie de découpage à deux niveaux a été ajoutée pour améliorer la relation entre une récupération précise et un contexte suffisant.

Chaque fragment enfant conservait un identifiant permettant de le relier à son parent. La recherche opérait sur les passages enfants, plus petits, tandis que le modèle de langage recevait les passages parents correspondants, plus vastes.

Cela a aidé à préserver le contexte environnant d’une information technique récupérée. Cette évolution répondait en partie à la récupération incomplète observée dans les questions multi-sujets.

TABLE 2 – Évolution des paramètres de découpage. Les valeurs sont exprimées en caractères.

Paramètre	Version initiale	Version ultérieure
Taille du fragment parent	1 500	1 500
Chevauchement parent	0	180
Taille du fragment enfant	250	400
Chevauchement enfant	70	80

Le chevauchement conserve une portion commune entre deux fragments voisins. Il vise à limiter la perte d’information lorsqu’une phrase ou une exigence se situe près d’une frontière de découpage. En contrepartie, il augmente la redondance et le nombre de fragments à indexer.

Cependant, la recherche parent/enfant ne garantit pas la couverture de tous les sujets : si aucun enfant pertinent n’est sélectionné pour une partie de la question, son parent peut toujours être absent.

8 Contrôle de la quantité de contexte

Au cours du développement, certaines réponses présentaient des omissions, des résumés inutiles, des détails non étayés ou des changements de langue.

Pour examiner ces problèmes, la configuration antérieure limitait le contexte à **quatre passages parents**, représentant environ **6 000 caractères de texte**, hors en-têtes de sources.

Le prompt a également été affiné pour demander des réponses concises et directes en français.

Ces modifications visaient à réduire le contexte non pertinent et à faciliter l’inspection manuelle des réponses. La surcharge de contexte et le positionnement de l’information étaient des causes

possibles des échecs, mais les expériences n'ont pas isolé de mécanisme spécifique du type *Lost in the Middle*.

Le budget de contexte a ensuite différé selon les configurations :

- **Mistral** : jusqu'à quatre passages parents ;
- **Qwen** : jusqu'à huit passages parents.

Cette distinction est importante dans la comparaison finale. Ces limites sont exprimées en nombre de passages et en caractères, et non directement en tokens. Elles ne représentent donc pas une mesure exacte de l'occupation de la fenêtre de contexte du modèle.

9 Ajout de la fusion de rangs réciproques

La fusion initiale basée sur la concaténation a été remplacée par la **fusion de rangs réciproques** (*Reciprocal Rank Fusion*, RRF).

La RRF combine les positions des passages dans les classements BM25 et sémantique. Un passage reçoit des contributions de chaque classement dans lequel il apparaît.

Dans l'implémentation, le score d'un passage d est :

$$S(d) = \sum_{r: d \in r} \frac{w_r}{60 + \text{rang}_r(d)}$$

Les rangs commencent à 1. Les poids attribués aux recherches lexicale et sémantique sont chacun fixés à 0,5. Un passage absent d'un classement ne reçoit pas de contribution de celui-ci.

La constante 60 modère les écarts de contribution entre les premières positions. Ces paramètres constituent les choix de configuration du prototype ; ils n'ont pas fait l'objet d'une optimisation systématique.

Cette méthode fournit un classement conjoint sans exiger que les scores BM25 et les scores de similarité vectorielle partagent la même échelle.

L'objectif était de réduire le biais d'ordonnancement de la fusion précédente et d'améliorer la sélection des candidats. La RRF reste une méthode de classement : elle ne garantit pas que chaque passage nécessaire parvienne au modèle de langage.

10 Ajout des noms de sources et de la pagination

La traçabilité des sources est devenue une exigence centrale à mesure que le prototype a évolué.

Lors de l'ingestion, les informations de page ont été enregistrées dans le champ de métadonnées `page_number`. Les versions ultérieures ont également inclus le nom du fichier source dans le contexte formaté.

Le prompt demandait au modèle de citer le document source et la page.

Cela facilitait l'audit des réponses, puisque l'utilisateur pouvait inspecter les passages présentés au modèle. Deux limites subsistaient :

- les citations générées pouvaient contenir un nom de fichier ou une référence incorrects ;
- les positions des pages du PDF pouvaient différer des numéros de page imprimés dans le document.

Par conséquent, les métadonnées des sources favorisent la traçabilité, mais les citations générées doivent toujours être vérifiées.

11 Migration de Mistral vers Qwen3

La configuration initiale utilisait **Mistral** pour la génération et **nomic-embed-text** pour les embeddings.

Une seconde configuration a été introduite avec :

- **Qwen3 8B**, via Ollama, pour la génération ;
- **Qwen3 Embedding 0.6B** pour la recherche sémantique ;
- ultérieurement, **Qwen3 Reranker 0.6B** pour le reclassement des candidats.

La configuration de génération visait une quantification Q4. Le tag du modèle installé et sa quantification exacte doivent être enregistrés depuis Ollama pour assurer la reproductibilité.

La migration a été motivée en partie par les difficultés rencontrées avec les formulations restrictives et l'attribution des acteurs, notamment la distinction entre le personnel autorisé à recevoir des livraisons et le personnel auquel cela est interdit.

Les prompts ont été affinés pour mettre l'accent sur les restrictions, les acteurs et les exigences spécifiques au document.

Ces modifications visaient à améliorer la qualité des réponses. Elles ne prouvent pas que les erreurs liées à la négation ont été éliminées, ni que Qwen est universellement meilleur pour interpréter les documents techniques français.

11.1 Construction du prompt technique

Dans la version finale, le prompt ne se limite pas à demander une réponse fondée sur les documents. Il demande aussi de distinguer les obligations, les recommandations et les possibilités, de signaler les ambiguïtés et de séparer les informations divergentes selon leur document d'origine.

Ces distinctions sont importantes pour l'usage visé. Une recommandation présentée comme une obligation, ou une exigence attribuée au mauvais chantier, peut conduire à une interprétation incorrecte du cahier des charges.

Le prompt indique également que les extraits constituent des sources documentaires et non des instructions à exécuter. Cette règle vise à limiter l'influence d'instructions éventuellement présentes dans les PDF. Elle constitue une précaution, mais sa résistance aux injections de prompt n'a pas été évaluée par une campagne de tests dédiée.

Le modèle est configuré avec **temperature=0** pour limiter la variabilité des réponses. La consigne **/no_think** est également présente pour demander l'absence de raisonnement étendu. Ces choix

ne garantissent ni l’exactitude factuelle ni une reproductibilité parfaite.

12 Ajout d’un reranker neuronal

La configuration Qwen a introduit une étape de classement supplémentaire après la recherche hybride et la RRF.

Le **Qwen3 Reranker 0.6B** évaluaient les paires question–passage pour estimer la pertinence des passages enfants candidats. Les enfants sélectionnés étaient ensuite rattachés à leurs passages parents.

Dans la version ultérieure de la configuration Qwen, les paramètres de recherche incluaient :

- jusqu’à 12 enfants par méthode de recherche ;
- jusqu’à 20 candidats pour le reclassement ;
- jusqu’à 8 enfants conservés après le reclassement ;
- jusqu’à 8 passages parents finaux.

Contrairement à la RRF, qui combine les positions de classement, le reranker évalue la relation entre la question et le texte de chaque candidat.

Cela a ajouté une évaluation de pertinence plus ciblée, mais a également introduit un temps de traitement supplémentaire. Sa contribution individuelle n’a pas été isolée par une expérience d’ablation dédiée.

12.1 Sélection des parents et limitation de la redondance

Après le reclassement des enfants, plusieurs résultats peuvent appartenir à un même parent. Envoyer plusieurs fois ce parent au modèle augmenterait inutilement la quantité de contexte. L’application regroupe donc les résultats par identifiant parent et conserve, pour chacun, l’enfant ayant obtenu le meilleur score de reranking.

Ce choix évite de favoriser un parent uniquement parce qu’il contient plusieurs fragments similaires. Il présente néanmoins une limite : plusieurs passages complémentaires associés au même parent n’augmentent pas son score, puisque seul le meilleur enfant détermine son classement.

TABLE 3 – Exemple fictif illustrant deux méthodes d’agrégation des scores des enfants.

Parent	Scores des enfants	Somme	Maximum
A	0,60 ; 0,58 ; 0,57	1,75	0,60
B	0,90	0,90	0,90

Dans cet exemple, une somme favoriserait le parent A, alors que la sélection par meilleur enfant placerait B en premier. Les valeurs servent uniquement à expliquer le mécanisme et ne correspondent pas à une mesure expérimentale.

Dans le code, les huit meilleurs enfants sont sélectionnés avant le regroupement par parent. Plusieurs enfants pouvant appartenir au même parent, le contexte final peut contenir moins de huit

parents distincts. Huit constitue donc une limite maximale, et non un nombre systématiquement atteint.

L'appel au reranker utilise une fonction sigmoïde. Toutefois, une valeur comprise entre 0 et 1 ne doit pas être interprétée comme une probabilité validée que la réponse finale soit correcte. Le score sert ici au classement des passages.

12.2 Gestion des ressources matérielles

L'exécution locale nécessite de limiter la charge des différentes étapes. Les embeddings sont calculés par lots de seize passages, tandis que le reranker traite les paires question–passage par lots de quatre.

Ce découpage vise à éviter l'envoi simultané de tout le corpus aux modèles. Il réduit la taille de chaque lot traité, sans supprimer le coût total du calcul sur l'ensemble des passages.

Le chargement du reranker privilégie MPS lorsque cette accélération est disponible sur la machine Apple Silicon. Si ce chargement échoue, le programme effectue une nouvelle tentative sur CPU.

Ce mécanisme concerne le chargement du reranker ; il ne garantit pas une récupération automatique après toute erreur survenant pendant les prédictions. Il est également distinct de la gestion matérielle des modèles exécutés par Ollama.

L'index documentaire et le reranker sont mis en cache afin de ne pas répéter leur préparation à chaque interaction. Ces dispositions permettent de distinguer le coût initial de préparation du coût de traitement d'une question. Leur effet individuel sur la mémoire et les performances n'a cependant pas été mesuré séparément.

13 Extension du prototype à plusieurs PDF

L'application mono-document initiale a été étendue pour charger tous les PDF du répertoire du projet.

Le corpus d'évaluation contenait quatre fichiers :

- 19082-COMP-CCTP-001.pdf ;
- cctp_lot_1.pdf ;
- cctp2.pdf ;
- cctp3.pdf.

Les noms des documents ont été conservés via le champ de métadonnées `source_name`. Les identifiants parents et enfants reliaient les passages récupérés à leurs documents sources.

Cela a permis de répondre aux questions sur l'ensemble du corpus et à l'interface d'afficher des références spécifiques aux documents.

Cependant, les documents concernent des projets différents. Une exigence trouvée dans un CCTP ne doit pas être automatiquement appliquée à un autre.

Les tests ont révélé que le modèle pouvait encore combiner des exigences non liées. La sélection des documents et la séparation des projets constituent donc des axes d'amélioration majeurs.

13.1 Gestion des changements du corpus

La préparation de l'index reçoit une signature des fichiers fondée sur leurs chemins, leurs tailles et leurs dates de modification. Cette signature participe à la clé de cache : une modification détectée entraîne une nouvelle préparation.

Une empreinte de cette signature intervient également dans le nom de la collection Chroma. Ce mécanisme vise à séparer les collections correspondant à des états différents du corpus.

Il s'agit d'une détection fondée sur les propriétés des fichiers, et non d'une comparaison exhaustive de leur contenu. Elle se distingue du calcul d'empreinte utilisé pour les expériences d'évaluation, qui lit le contenu des PDF.

13.2 Architecture de traitement d'une question

La préparation du corpus est réalisée en amont : extraction du texte, découpage, construction de BM25 et indexation vectorielle. Lors d'une question, les deux moteurs interrogent les fragments enfants déjà préparés.

La figure suivante présente le cheminement d'une question dans la configuration Qwen.

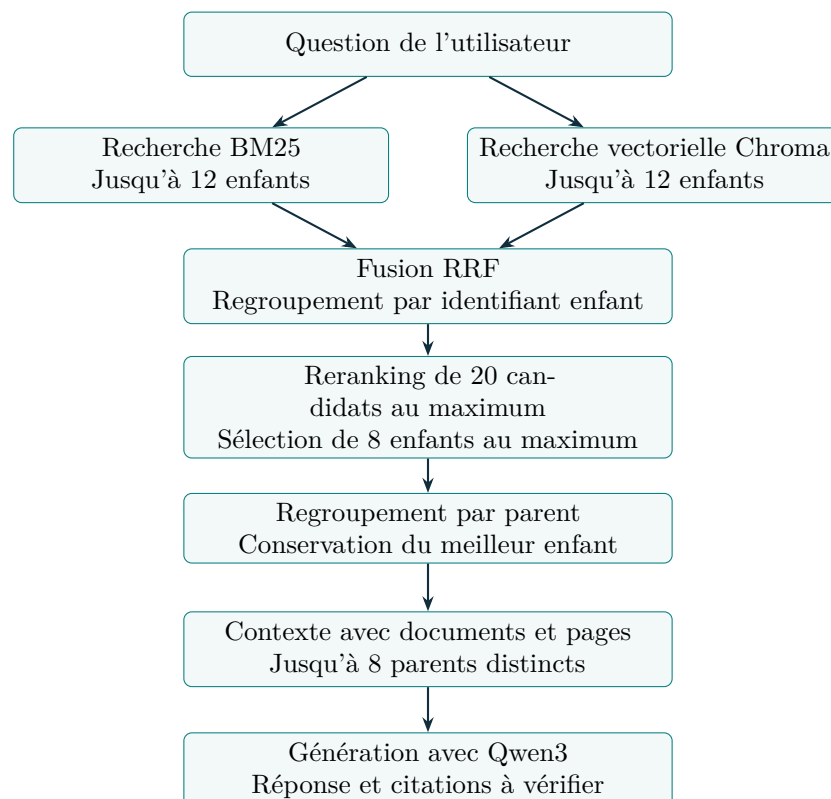


FIGURE 1 – Chaîne de traitement d'une question dans la configuration Qwen. Les nombres indiquent des maxima.

14 Évaluation automatique avec RAGAS

Un pipeline d'évaluation automatique a été mis en place avec **RAGAS 0.4.3** et **Llama 3.1 8B** comme juge local.

Le jeu de données contenait :

- 20 questions avec des réponses attendues ;
- 5 questions désignées comme impossibles à traiter à partir des informations du corpus.

Les réponses de référence ont d'abord été générées avec Gemini. Elles constituaient un jeu de données de référence provisoire et nécessitent une vérification humaine à partir des PDF.

Les deux configurations ont été évaluées à l'aide des mêmes questions, réponses de référence, modèle juge et script d'évaluation. L'ensemble des 25 enregistrements a été évalué sans qu'aucune erreur d'exécution des métriques soit signalée.

La comparaison concerne des **configurations RAG complètes**, plutôt que des modèles de génération isolés. Les composants de recherche différaient et Qwen a reçu entre quatre et huit passages, tandis que Mistral en a reçu quatre.

Préparation et contrôle du jeu de données

Le fichier d'évaluation contient sept champs, présentés dans le tableau suivant.

TABLE 4 – Structure du fichier de questions de référence.

Champ	Rôle
id	Identifiant unique de la question.
question	Question transmise au système.
reference_answer	Réponse attendue pour l'évaluation.
expected_document	Document de référence attendu.
expected_pages	Pages attendues, par exemple 14 ou 14 15.
answerable	Indication oui/non sur la possibilité de répondre.
category	Catégorie associée à la question.

La fonction de lecture vérifie la présence des colonnes attendues, l'unicité des identifiants et l'absence de questions vides. Pour les questions déclarées répondables, elle exige une réponse de référence, un document et des pages attendues.

Un nettoyage retire également les balises de citation issues de Gemini et corrige un cas identifié de séparateur supplémentaire. Ces traitements vérifient la structure du fichier, mais ne valident pas l'exactitude des références : cette dernière nécessite une lecture des documents.

Pendant l'exécution du RAG, seule la question est transmise à la recherche. La réponse de référence, le document attendu et les pages attendues ne sont pas utilisés pour guider la génération. Cette séparation évite de fournir au système les réponses qu'il doit retrouver.

Collecte des réponses et reprise des expériences

L'évaluation est organisée en deux étapes. L'application exécute les questions et exporte les réponses avec leurs contextes. Un script distinct utilise ensuite RAGAS pour calculer les métriques.

Cette séparation permet d'examiner ou de réévaluer les réponses sauvegardées sans relancer toute la génération.

Chaque résultat est immédiatement ajouté à un fichier JSONL, contenant un objet JSON par ligne. Il conserve notamment la question, la réponse, les passages récupérés, les sources, la durée et les éventuelles erreurs.

Lors d'une reprise, les questions déjà exécutées avec succès sont ignorées et les questions en erreur sont retentées. Une erreur sur une question n'interrompt donc pas nécessairement le traitement des suivantes.

Le statut `ok` indique uniquement que l'exécution s'est terminée sans erreur technique ; il ne signifie pas que la réponse est correcte.

Une empreinte SHA-256 calculée à partir du script, du fichier de questions et du contenu des PDF identifie l'expérience. Elle permet de séparer les résultats lorsque ces éléments changent.

Cette empreinte ne suffit cependant pas à décrire l'ensemble de l'environnement : les versions des dépendances, des modèles et les caractéristiques matérielles doivent également être documentées pour une reproduction complète.

14.1 Résultats

TABLE 5 – Évaluation automatique des deux configurations RAG.

Critère évalué	Mistral	Qwen3
Fidélité aux extraits récupérés	0,547	0,680
Exactitude factuelle – F1	0,571	0,583
Précision du contexte	0,865	0,897
Rappel du contexte	0,936	1,000
Abstention correcte selon le juge	4/5	5/5
Temps de réponse moyen par question	21,7 s	56,6 s

Les quatre premières métriques ont été calculées sur les 20 questions auxquelles il était possible de répondre. L'abstention a été évaluée séparément sur les cinq questions sans réponse attendue dans le corpus.

Les mesures du temps de réponse incluent la recherche, le reclassement, le cas échéant, et la génération. Elles excluent l'indexation initiale et le chargement des modèles.

Qwen a obtenu des scores automatiques moyens plus élevés, tandis que Mistral a été environ **2,6 fois plus rapide**. La différence d'exactitude factuelle était faible : **0,012**.

Ces valeurs sont des scores d'évaluation et non des pourcentages de questions ayant reçu une réponse correcte.

14.2 Analyse des réponses individuelles

Question 15 : documents sur l’amiante

Qwen a mentionné les documents relatifs à l’amiante, tandis que Mistral a produit une réponse concernant la documentation électrique.

Le contexte de Mistral ne contenait pas l’information pertinente sur l’amiante. Cela illustre un échec de la recherche et de la réponse dans la configuration Mistral, sans isoler la contribution du modèle de génération lui-même.

Question 11 : référence exacte à l’article

Mistral a correctement extrait la référence **L.342-2**, alors que Qwen a déclaré qu’aucun article spécifique n’était fourni.

Cela démontre que les scores moyens plus élevés de Qwen ne se sont pas traduits par de meilleures réponses à chaque question.

Question 7 : mélange de projets différents

Qwen a correctement mentionné la galvanisation à chaud, mais a ajouté une exigence de protection incendie provenant d’un autre CCTP.

L’information supplémentaire risquait d’induire une confusion entre des exigences appartenant à des projets différents. La réponse de Mistral était plus ciblée, bien que sa citation générée ait été défectueuse.

14.3 Limites du juge local

Le juge était distinct des modèles évalués, mais n’était pas supposé être impartial ou infaillible.

Plusieurs résultats étaient discutables :

- Qwen a renvoyé **110 cm** pour la question 13, mais a reçu un score d’exactitude factuelle de **0,40**.
- Mistral a reçu un score de fidélité de **zéro** pour les questions 7, 14 et 19, bien que l’information centrale soit présente dans les extraits récupérés.
- Pour la question 25, Mistral a déclaré qu’aucune marque de peinture n’était spécifiée, mais le juge a attribué un score d’abstention de **zéro**.

Ces écarts illustrent pourquoi les scores automatiques doivent être complétés par des évaluations humaines de l’exactitude des réponses, de la validité des citations et de leur pertinence.

La taille réduite du jeu de données, les références provisoires et les configurations de recherche différentes limitent la portée des conclusions générales.

En particulier, le rappel de contexte égal à 1,000 pour Qwen est un résultat obtenu par ce juge sur cet échantillon. Il ne signifie pas que le système retrouve toujours toutes les informations nécessaires sur n’importe quel document.

15 Améliorations finales de l'interface

Après l'évaluation initiale, l'interface Streamlit a été repensée pour améliorer la présentation et l'ergonomie.

L'interface mise à jour a introduit :

- un design cohérent, bleu marine et turquoise ;
- des onglets séparés pour l'assistant, les documents et l'évaluation ;
- la persistance des réponses lors des interactions avec l'interface ;
- l'accès aux passages récupérés dans leur intégralité ;
- le téléchargement des PDF et l'exportation des réponses au format texte.

Les fonctions RAG ont été préservées durant cette refonte.

L'outil d'évaluation identifie les expérimentations en partie grâce à une empreinte numérique du script de l'application.

Par conséquent, une modification purement liée à l'interface crée un nouvel identifiant d'expérience si l'évaluation est relancée, même lorsque les fonctions de recherche et de génération restent inchangées.

15.1 Conservation des réponses pendant la session

Le dernier résultat est enregistré dans `st.session_state`. Cela permet de conserver la question, la réponse et les passages associés lors des réexecutions de l'interface provoquées par les interactions avec les widgets.

Ce mécanisme concerne l'état de la session utilisateur. Il ne constitue pas un historique durable des conversations et ne doit pas être confondu avec les fichiers de résultats enregistrés par le module d'évaluation.

15.2 Limites techniques et améliorations envisagées

L'examen de l'implémentation met en évidence plusieurs limites complémentaires aux erreurs observées dans les réponses.

Premièrement, aucune étape d'OCR n'est prévue explicitement. Les documents numérisés sans couche texte nécessiteraient donc un traitement supplémentaire avant leur indexation.

Deuxièmement, le nom d'un document mentionné dans une question n'active pas de filtre strict sur ce fichier. La recherche reste effectuée sur l'ensemble du corpus. Un filtre explicite par document ou par projet permettrait de mieux maîtriser le périmètre interrogé et de réduire les mélanges entre CCTP.

Troisièmement, aucun seuil minimal de pertinence n'est appliqué avant la génération. Le système retient les passages les mieux classés parmi ceux disponibles, même lorsqu'ils ne permettent pas de répondre. La décision de reconnaître l'absence d'information repose donc principalement sur le modèle et sur les instructions du prompt.

Enfin, le traitement des documents et l'inférence sont configurés localement, mais l'interface importe des polices depuis Google Fonts. Cette dépendance ne démontre pas un transfert du contenu des PDF ; elle signifie néanmoins que l'interface comporte encore une ressource externe. Pour une utilisation entièrement hors ligne, cette ressource devrait être remplacée par une police système ou hébergée localement.

16 Conclusion et perspectives

Le projet a évolué d'une expérimentation mono-document en ligne de commande vers une application locale multi-documents avec recherche hybride, récupération de contexte parent/enfant, métadonnées des sources et pipeline d'évaluation automatique.

Le processus de développement a révélé plusieurs défis pratiques : la récupération de termes techniques exacts, le contexte incomplet pour les questions multi-sujets, l'interprétation des restrictions, l'attribution des sources et le mélange d'exigences provenant de projets différents.

Sur le jeu de données d'évaluation de 25 questions, la configuration Qwen a obtenu des scores automatiques moyens plus élevés, tandis que Mistral a fourni des temps de réponse considérablement plus courts.

Qwen a donc été retenu **provisoirement comme configuration principale**, Mistral étant conservé comme base de comparaison plus rapide.

Le prototype n'élimine pas les hallucinations et ne garantit pas l'exactitude des citations. Ses résultats nécessitent une vérification avant toute utilisation technique professionnelle.

Après le téléchargement des modèles, la chaîne de traitement configurée fonctionne localement. Une évaluation de la confidentialité au niveau du déploiement nécessiterait également d'examiner l'environnement d'exploitation dans son ensemble.

Les prochaines étapes prioritaires seraient de vérifier humainement les références et les réponses, de mettre en place une sélection explicite des documents et de comparer les modèles génératifs sur des contextes strictement identiques.

Des expériences séparées permettraient ensuite de mesurer l'apport du reranker, de la fusion RRF et du nombre de passages transmis au modèle. L'utilisation de nouveaux documents et de nouvelles questions permettrait d'examiner si les observations réalisées se maintiennent au-delà du jeu d'évaluation initial.